

Práctica 1 de Recuperación de Información

Adquisición y procesamiento de información textual

Erik Figueiral Alonso · erik.figueiral.alonso@uvigo.gal

18 de septiembre · Práctica de dos semanas · Entrega: 2 de octubre

Horario · Erik Figueiral Alonso

Lunes **09:00–10:30**

Jueves **15:00–16:30**

Laboratorio de Ciencia Computacional

3 horas concertadas · erik.figueiral.alonso@uvigo.gal

Una búsqueda sencilla puede fallar

Empezamos con una necesidad de información, no con un algoritmo.

D1 «La CHICA repara televisores.»

D2 «Un taller vende automóviles.»

Consulta: «reparar coche»

D1 coincide en la acción; D2, en el objeto si ampliamos «coche». ¿Cuál sirve?

Qué estudia la Recuperación de Información

Organizar y representar documentos para acceder a la información que necesita una persona.

Documentos

Texto, imágenes, audio o vídeo.

Representación

Términos, campos o vectores.

Consulta y respuesta

Candidatos ordenados por utilidad estimada.

El objetivo es satisfacer una necesidad, aunque las palabras de la consulta no coincidan exactamente.

Hoy: de la Web al texto preparado

- 1 Adquirir** Obtener documentos de una fuente: Web, API o carga manual.
- 2 Procesar** Extraer texto y decidir sus unidades: tokens, normalización, filtrado.
- 3 Representar** Preparar términos o vectores para comparar documentos y consultas.

La construcción y el almacenamiento del índice se desarrollarán más adelante.

Buscar información es una necesidad antigua

De catálogos de biblioteca a documentos publicados y enlazados en la Web.



La World Wide Web, propuesta por Tim Berners-Lee en 1989, cambió la escala del problema.

ACM SIGIR es una conferencia internacional central para la investigación en RI.

Recuperación de datos y RI: dos preguntas distintas

Consulta estructurada

«Muéstrame registros cuya fecha_accidente esté entre 1910 y 1989». El campo y la condición están definidos.

Necesidad de información

«Encuentra documentos que expliquen accidentes ferroviarios de ese período». El texto puede expresarlo de muchas maneras.

En un sistema real podemos combinar filtros sobre campos con búsqueda textual.

Recuperar candidatos no basta: hay que ordenarlos



Ranking estima qué documentos serán más útiles para esta necesidad concreta.

Pregunta: ¿cambiaría el resultado con el tiempo, el lugar o la intención del usuario?

Qué consideraremos un documento

Un documento reúne contenido, estructura y metadatos.

Texto + estructura + otros medios

La estructura puede ser implícita (títulos y párrafos reconocibles) o explícita (etiquetas HTML/XML).

En nuestro notebook

id · título · URL
texto extraído

El formato influye en cómo se presenta y en qué significado podemos extraer.

La consulta también necesita interpretación

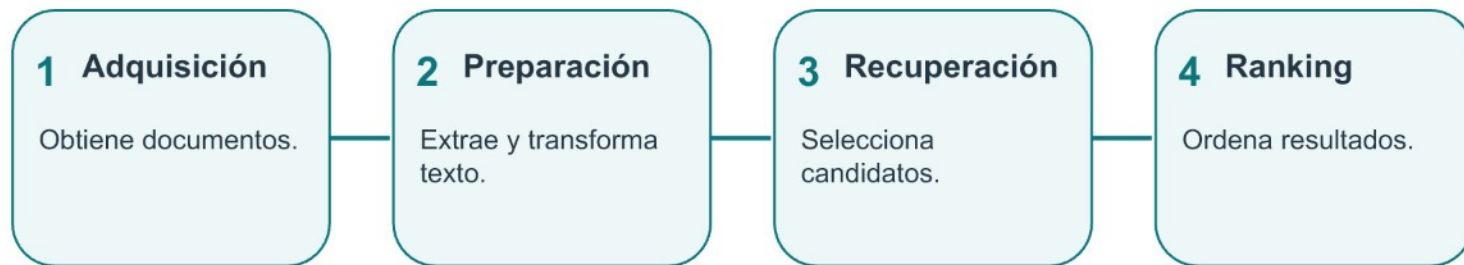
**Necesidad: documentos sobre
reparación de automóviles**

**Consulta inicial: «arreglar
coche»**

La primera formulación puede fallar. Podemos corregirla, ampliarla o buscar de nuevo tras leer resultados.

Ejemplo: «arreglar» → «reparar»; «coche» → «automóvil». La expansión puede añadir ruido.

El recorrido completo del sistema



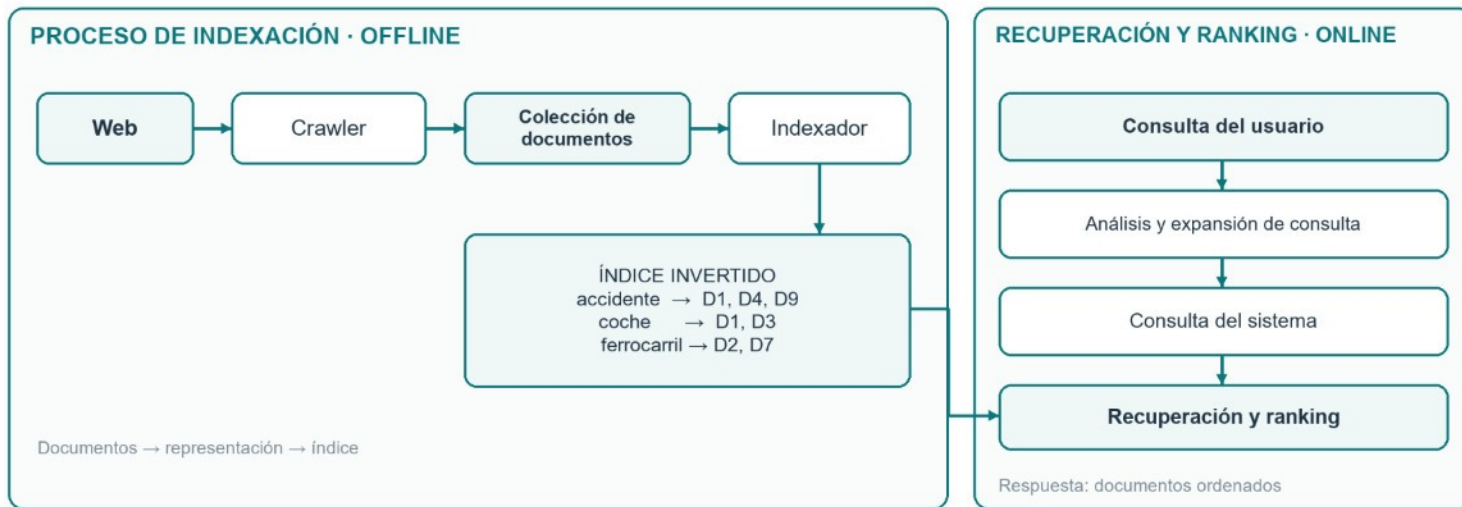
Los dos primeros preparan la colección. La consulta activa los dos últimos.

Arquitectura de un sistema de RI



Consulta y documentos deben transformarse de forma compatible. La expansión es opcional.

Del índice a los resultados · procesos



Una forma de adquirir documentos: el crawler

Un crawler (o spider) recorre enlaces para descubrir y descargar páginas.

Crawler

Descubre URLs y decide cuáles visitar.

Scraper

Extrae contenido útil del HTML descargado.

API

Entrega contenido estructurado cuando existe una interfaz oficial.

El crawler alimenta la colección; no construye por sí solo el índice de búsqueda.

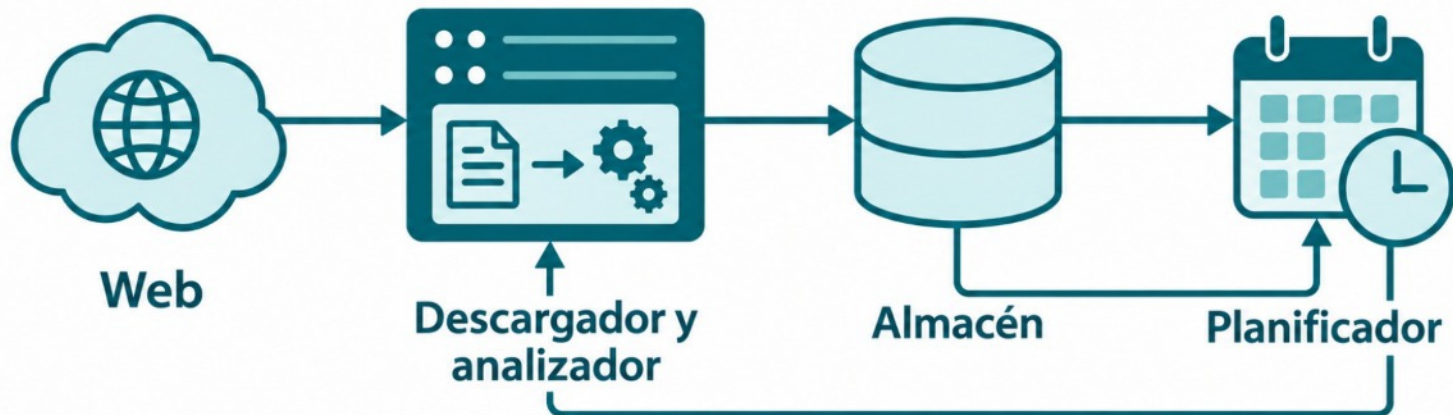
Cómo recorre la Web un crawler básico

La frontera es la cola de URLs pendientes.

- 1 Empezar** Añadir una o varias URL semilla.
- 2 Elegir** Sacar una URL permitida de la frontera.
- 3 Descargar** Obtener la página y registrar errores.
- 4 Parsear** Extraer texto, estructura y enlaces.
- 5 Repetir** Añadir enlaces nuevos y volver a 2; parar al cumplir un criterio.

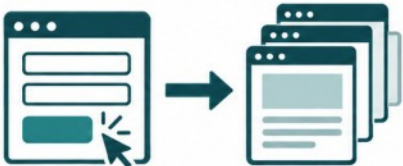
Evitar duplicados y limitar profundidad, volumen y frecuencia de solicitudes.

Arquitectura mínima de un crawler



El planificador envía URL al descargador; el almacén conserva páginas y metadatos.

Qué parte de la Web puede recorrer

	Estática	Dinámica
Pública		
Privada		

Los enlaces no descubren por sí solos páginas tras formularios o autenticación.

Qué URL visita primero

El planificador no puede descargar toda la Web: elige prioridades y un criterio de parada.

Cobertura

Visitar más páginas y descubrir nuevas zonas.

Calidad

Priorizar páginas relevantes para un tema.

Frescura

Volver a páginas que pueden haber cambiado.

General: amplía cobertura. Focalizado: sigue un tema. Ambos necesitan límites y cortesía.

Antes de descargar: consultar robots.txt

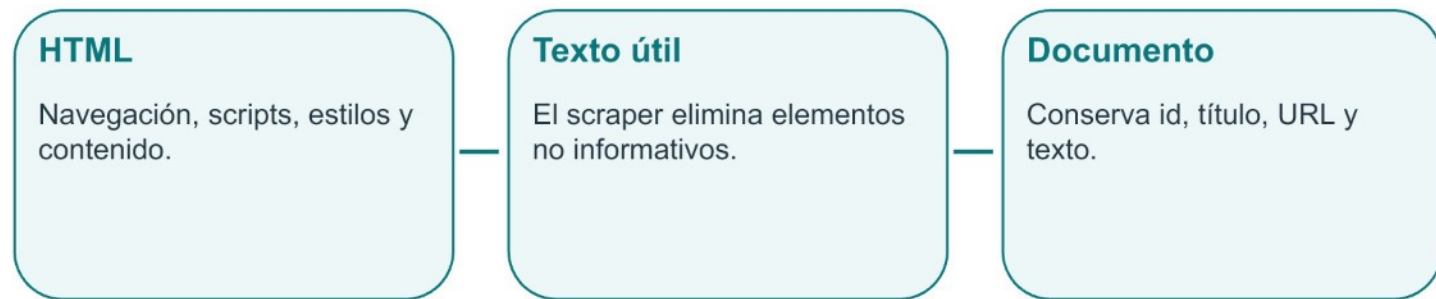
```
User-agent: *  
Disallow: /privado/  
Allow: /privado/info-publica.html
```

¿Puede visitarse [/privado/info-publica.html](#)?

El archivo expresa reglas de rastreo para agentes identificados. No protege información confidencial ni garantiza que una URL no aparezca en resultados.

En el notebook: interpretar reglas y comprobarlas con `urllib.robotparser`.

De la página descargada al documento



En la práctica también obtendremos documentos mediante una API de Wikipedia.

Del texto bruto a una representación para buscar

D1 «La CHICA repara televisores.»

Texto → **tokens** → **normalización** → **filtrado** → **variantes morfológicas** →
términos

Cada decisión modifica lo que podremos buscar después. Seguiremos el mismo documento.

Tema 2: diaps. 47 y 56.

Tokenizar: decidir cuáles son las unidades

«La CHICA repara televisores.»

[La] [CHICA] [repara] [televisores]

Antes de tokenizar HTML, extraemos el texto visible. La puntuación, fechas y guiones exigen reglas.

Una expresión puede contener varios tokens

«inteligencia artificial»

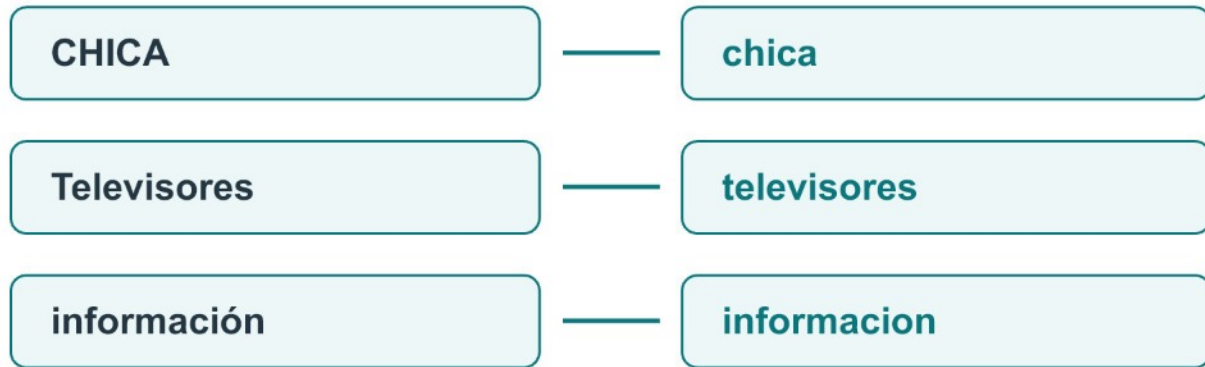
Tokens: inteligencia | artificial

Término de dos palabras

Exige contigüidad y orden.

Dos palabras presentes en lugares distintos del documento no equivalen a la expresión.

Normalizar: hacer comparables variantes escritas



Minúsculas suele ayudar. Eliminar tildes depende de la colección y puede perder distinciones.

D1 ahora: [la, chica, repara, televisores].

Stopwords: filtrar con un objetivo concreto

D1: [la, chica, repara, televisores]

[chica, repara, televisores]

«La» se elimina; la decisión depende de la tarea. En «el coche no funciona», «no» sí cambia el sentido.

Tema 2: diaps. 61 y 63. Notebook: comparar antes y después.

Stemming y lematización: agrupar variantes

Entrada tras stopwords: [chica, repara, televisores]

Stemming

Recorta terminaciones con reglas. Puede producir raíces que no son palabras; la salida depende del algoritmo.

Lematización

Busca la forma de diccionario. Ejemplo ilustrativo: [chica, reparar, televisor].

Objetivo: mejorar coincidencias. Riesgo: perder distinciones útiles.

Qué términos conservamos para buscar

D1: «La CHICA repara televisores.» → {chica, reparar, televisor}

Política amplia

Conservar términos informativos, incluidos verbos.

Política selectiva

Priorizar nombres o entidades según la tarea.

Si quitamos «reparar», perdemos la única coincidencia de D1 con la consulta.

Un recurso léxico puede ampliar la consulta

Tesauro o diccionario: relaciones explícitas entre términos.

Sinonimia

coche ↔ automóvil

Puede hacer coincidir D2 por el objeto.

Jerarquía

automóvil → vehículo

«Vehículo» es más amplio que «automóvil».

Consulta «reparar coche»: en una búsqueda flexible, ampliar «coche» puede recuperar D2 por objeto, pero D2 vende. Más candidatos no significa más relevancia.

Representación dispersa y densa

Dispersa

Una dimensión por término del vocabulario. Casi todos los pesos son cero.

D1: {chica:1, reparar:1, televisor:1}

Densa

Un modelo aprende un vector compacto de números. Términos distintos pueden quedar próximos según el contexto.

D1: [0,21; -0,34; ...] (ficticio)

Esta práctica trabaja sobre todo con el recorrido textual y disperso.

La consulta debe ser compatible con los documentos

D1 $\rightarrow$ {chica, reparar, televisor} · D2 $\rightarrow$ {taller, vender, automóvil}

Consulta «reparar coche» $\rightarrow$ {reparar, coche, automóvil*}

*Automóvil aparece solo si expandimos la consulta. D1 coincide en la acción; D2, en el objeto. Hay que estimar la relevancia.

Ahora, en el notebook

- 1 Preparación** Abrir el notebook en Colab y comprobar los recursos.
- 2 Adquisición** robots.txt, scraping y documentos desde la API.
- 3 Procesamiento** Segmentación, normalización, tokenización, stopwords y stemming.
- 4 Representación** Palabras y n-gramas; comparar las unidades obtenidas.

Siguiente sesión: frecuencias y ley de Zipf.

Entrega en Moovi: 2 de octubre

Índice · Ley de Zipf

¿Por qué necesitamos estudiar la distribución de las palabras?

- | | | |
|---|----------------------------|---|
| 1 | Medir | Frecuencia absoluta y relativa. |
| 2 | Ordenar | Rango y ley de Zipf. |
| 3 | Interpretar | Exponente, límites y consecuencias para RI. |
| 4 | Completar el modelo | Heaps, probabilidades y entropía. |

La distribución determina qué términos discriminan, cuánto ocupa el vocabulario y cómo conviene almacenar el índice.

Primero medimos: frecuencia absoluta y relativa

Frecuencia absoluta $f(t,d)$

Número de veces que aparece el término t en el documento d .

Documento A · 100 tokens

«recuperación» aparece 10 veces

$$f = 10 \quad \cdot \quad p = 10/100 = 0,10$$

Frecuencia relativa $p(t,d)$

Proporción del documento ocupada por t : $f(t,d) / |d|$.

Documento B · 1000 tokens

«recuperación» aparece 50 veces

$$f = 50 \quad \cdot \quad p = 50/1000 = 0,05$$

B contiene más apariciones; en A el término tiene más peso proporcional.

Después ordenamos: rango y ley de Zipf

El término más frecuente tiene rango 1; el siguiente, rango 2; y así sucesivamente.

$$f(r) \approx C r^{-\alpha} = \frac{C}{r^{\alpha}}$$

rango r	1	2	3	4
f(r)	1000	500	333	250

Con $\alpha = 1$ y $C = 1000$, la frecuencia aproximada es $1000/r$.

C · escala

$C \approx f(1)$: frecuencia del término que ocupa el primer rango.

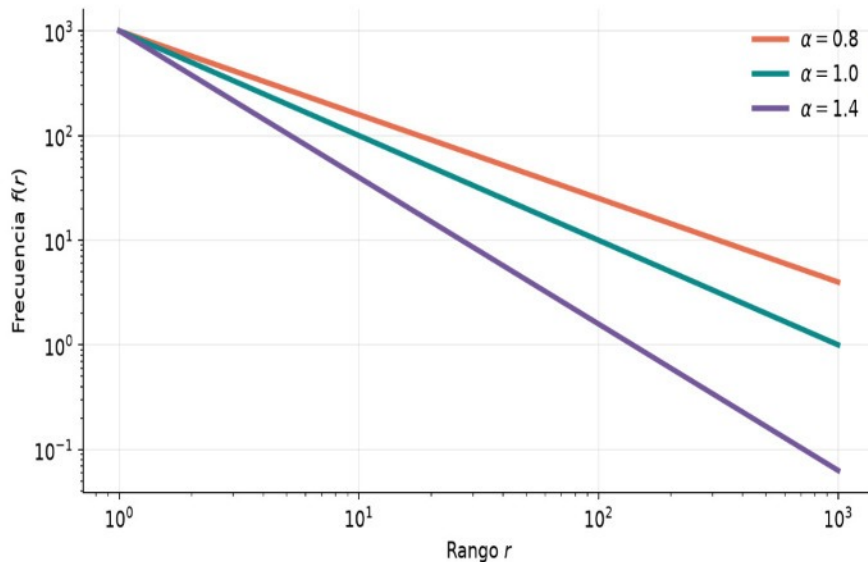
α · ritmo de caída

α grande: las frecuencias disminuyen más deprisa al aumentar el rango.

Regularidad empírica

Resume la forma global. No predice exactamente cada término.

En escala log-log, α es la pendiente



$$\log f(r) \approx \log C - \alpha \log r$$

pendiente = $-\alpha$

$\alpha = 0,8$ Las frecuencias decrecen lentamente.

$\alpha = 1,4$ Los términos de rangos altos se vuelven raros más deprisa.

Normalizar Zipf: las frecuencias deben sumar n

Si el corpus tiene n tokens y V términos distintos:

$$H_V(\alpha) = \sum_{i=1}^V \frac{1}{i^\alpha}$$

$$f_i \approx \frac{n}{i^\alpha H_V(\alpha)}$$

¿Para qué sirve $H_V(\alpha)$?

Es una constante de normalización: reparte los n tokens entre los V rangos para que las frecuencias sumen aproximadamente n.

Ejemplo · V=4, $\alpha=1$

$H_4 = 1 + 1/2 + 1/3 + 1/4 \approx 2,08$.

Si n=2080: $f_1 \approx 1000$ y $f_2 \approx 500$.

Con $\alpha=1$, H_V crece aproximadamente como $\ln V$. Con $\alpha>1$, la suma se aproxima a un límite.

Tres frecuencias: tf, cf y df

Ejemplo: tres documentos; «recuperación» aparece 3 veces en D1, 0 en D2 y 2 en D3.

$$\mathbf{tf(t,D1) = 3}$$

Frecuencia del término dentro de un documento. Sirve para estimar su importancia local.

$$\mathbf{cf(t) = 5}$$

Frecuencia en la colección: 3+0+2 apariciones. La práctica utiliza cf para la curva de Zipf.

$$\mathbf{df(t) = 2}$$

Frecuencia documental: aparece en D1 y D3. Cada documento cuenta una sola vez.

$$idf(t) = \log \frac{N}{df(t)}$$

Más adelante: si un término aparece en pocos documentos, IDF le asigna más peso. Bajo un modelo Zipfiano de df, el rango y el IDF quedan relacionados.

Zipf ayuda, pero no describe todo el lenguaje

Orden y contexto

Zipf cuenta palabras, pero no distingue «perro muerde hombre» de «hombre muerde perro».

Distribución entre documentos

Una palabra puede concentrarse en pocos documentos; una distribución uniforme sería irreal.

Stopwords

Ocupan los primeros rangos y discriminan poco, pero eliminarlas a ciegas rompe «The Who».

Cabeza y cola larga

La cabeza contiene pocos términos muy frecuentes. La cola reúne muchísimos términos de rango alto, cada uno con una o pocas apariciones.

La binomial negativa puede modelar el agrupamiento de apariciones entre documentos. Heaps modela el crecimiento del vocabulario.

Heaps completa la imagen: el vocabulario sigue creciendo

$$V(n) = K n^{\beta}$$

Qué significa

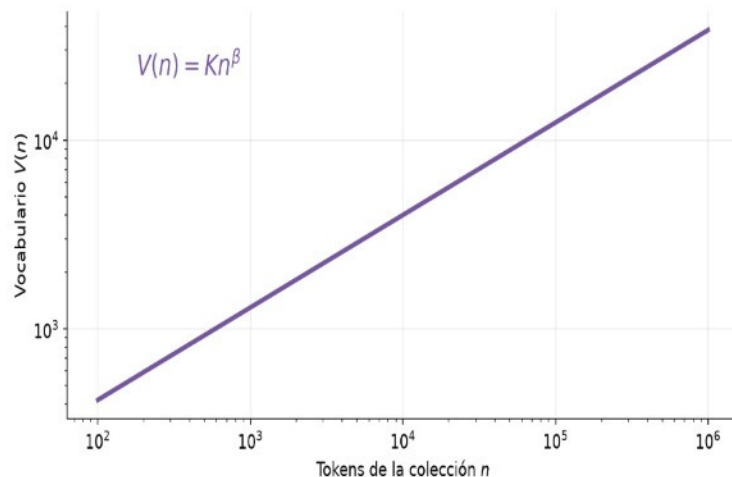
n : tokens observados

$V(n)$: términos distintos

K y β : parámetros estimados en el corpus

$0 < \beta < 1 \Rightarrow$ crecimiento sublineal

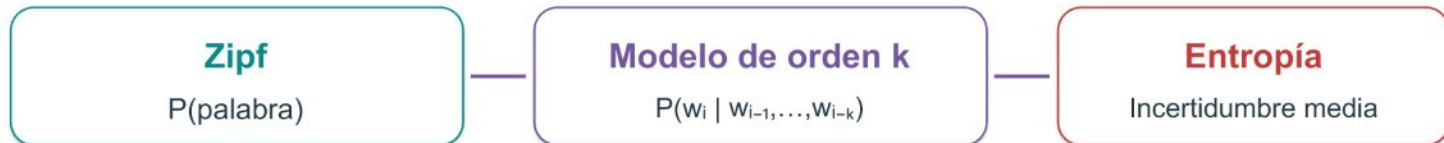
Valores observados habitualmente en TREC-2: $\beta \approx 0,4-0,6$.



**Duplicar el texto no duplica el vocabulario:
aparecen palabras nuevas, pero cada vez más
despacio.**

De Zipf al modelado del lenguaje

Zipf modela probabilidades marginales; un modelo del lenguaje añade contexto.



$$H(X) = - \sum_{x \in \mathcal{A}} p(x) \log_2 p(x)$$

Ejemplo · A A A B

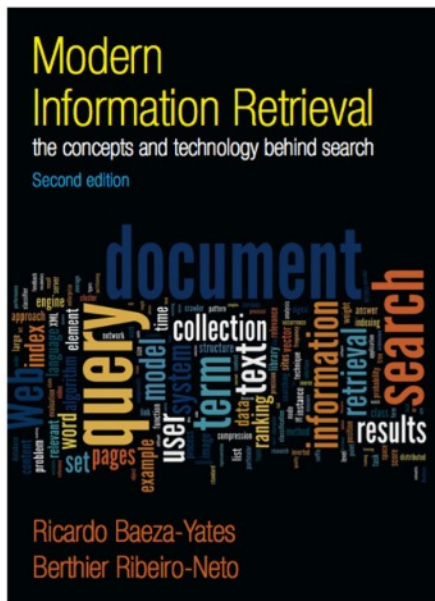
$p(A)=3/4$, $p(B)=1/4$

$I(A)=0,42$ bits; $I(B)=2$ bits

$H=0,81$ bits por símbolo

Zipf explica qué palabras son globalmente frecuentes. Markov, estados finitos o gramáticas modelan dependencias. La entropía resume cuánta incertidumbre queda.

Referencia principal de esta introducción



Modern Information Retrieval

The Concepts and Technology Behind Search
2.^a edición

Ricardo Baeza-Yates
Berthier Ribeiro-Neto

Base para la visión del sistema, los documentos, el crawling y el procesamiento textual.

Portada: sitio web oficial de los autores.